

AI-Assisted API Development

Task Management System — RESTful API

1. The Prompt

The following prompt was used inside Claude (Cowork) to generate the initial API scaffold:

"Act as a Senior .NET Architect. Generate a production-ready RESTful API in .NET 9 for a Task Management System following Clean Architecture and Domain-Driven Design (DDD) principles. I need a RESTful API for a task management system using .NET 9 and Clean Architecture. Each task has: title (required, max 200 chars), description (optional), status (Pending / InProgress / Done), and due_date. Tasks belong to a User — assume the User entity and JWT auth are already in place. I need full CRUD. Protect create, update and delete behind [Authorize]. Return 404 when a task is not found and 403 when a user tries to touch someone else's task. Keep it simple — no CQRS, just service + repository pattern."

2. Generated Output

Task Entity (Domain Layer)

```
public class Task : BaseEntity
{
    public string Title { get; private set; } = string.Empty;
    public string? Description { get; private set; }
    public TaskStatus Status { get; private set; } = TaskStatus.Pending;
    public DateTime? DueDate { get; private set; }
    public Guid UserId { get; private set; }

    private Task() { }

    public static Task Create(string title, string? description,
        DateTime? dueDate, Guid userId)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new DomainException("Title is required.");
        if (title.Length > 200)
            throw new DomainException("Title must not exceed 200 characters.");
        if (dueDate.HasValue && dueDate.Value < DateTime.UtcNow)
            throw new DomainException("Due date cannot be in the past.");
        if (userId == Guid.Empty)
            throw new DomainException("UserId is required.");

        return new Task {
            Title = title, Description = description,
            DueDate = dueDate, UserId = userId
        };
    }

    public void Update(string title, string? description, DateTime? dueDate)
    {
        if (string.IsNullOrWhiteSpace(title))
            throw new DomainException("Title is required.");
        Title = title; Description = description; DueDate = dueDate;
    }

    public void ChangeStatus(TaskStatus status) => Status = status;
}
```

Tasks Controller (API Layer)

```
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class TasksController : ControllerBase
{
    private readonly ITaskService _taskService;
    public TasksController(ITaskService taskService) =>
        _taskService = taskService;

    private Guid GetUserId() =>
        Guid.Parse(User.FindFirstValue(ClaimTypes.NameIdentifier!));

    [HttpGet]
    public async Task<ActionResult> GetAll() =>
        Ok(await _taskService.GetAllAsync(GetUserId()));

    [HttpGet("{id:guid}")]
    public async Task<ActionResult> GetById(Guid id)
    {
        var task = await _taskService.GetByIdAsync(id, GetUserId());
        return task is null ? NotFound() : Ok(task);
    }

    [HttpPost]
    public async Task<ActionResult> Create([FromBody] SaveTaskDto dto)
    {
        var task = await _taskService.CreateAsync(dto, GetUserId());
        return CreatedAtAction(nameof(GetById), new { id = task.Id }, task);
    }

    [HttpPut("{id:guid}")]
    public async Task<ActionResult> Update(Guid id, [FromBody] SaveTaskDto dto)
    {
        try { return Ok(await _taskService.UpdateAsync(id, dto, GetUserId())); }
        catch (KeyNotFoundException) { return NotFound(); }
        catch (UnauthorizedAccessException) { return Forbid(); }
    }

    [HttpDelete("{id:guid}")]
    public async Task<ActionResult> Delete(Guid id)
    {
        try { await _taskService.DeleteAsync(id, GetUserId()); return NoContent(); }
        catch (KeyNotFoundException) { return NotFound(); }
        catch (UnauthorizedAccessException) { return Forbid(); }
    }
}
```

3. Validating the AI Output

The generated code was reviewed and validated across four dimensions:

Architectural correctness

Verified that dependency direction respected Clean Architecture rules: Domain had no external references, Application depended only on Domain interfaces, and Infrastructure was wired through DI in Program.cs.

Domain invariants

Confirmed that entity creation went through factory methods (Task.Create()) with private setters, preventing invalid state from being set externally. The status enum was validated to include only the three expected values.

Authorization model

Checked that every write operation (create, update, delete) extracted the userId from the JWT claims and passed it down to the service layer, which performed the ownership check before executing the operation.

Test coverage

Unit tests were written for the service layer using xUnit + Moq + FluentAssertions, covering: happy paths, not-found scenarios, and unauthorized access attempts by a different user.

4. Corrections and Improvements

Several adjustments were made to the initial output:

Due date validation

The AI did not include a check for past due dates. This was added to Task.Create() as a domain rule since accepting a task with a due date in the past makes no business sense.

Status change as separate method

The initial output allowed status to be changed through Update(). This was separated into a dedicated ChangeStatus() method to make intent explicit and allow for future workflow rules (e.g., a task in Done cannot go back to Pending).

HTTP 403 vs 404 on unauthorized access

The first version returned 404 for both "not found" and "wrong owner" cases. This was corrected to return 403 (Forbidden) when the task exists but belongs to a different user, and 404 only when the task genuinely does not exist.

5. Edge Cases and Authentication

Authentication

All endpoints except GET (list and detail) require a valid JWT Bearer token. The token is validated by the ASP.NET Core JWT middleware configured in Program.cs. The userId is extracted from the NameIdentifier claim and never trusted from the request body.

Input validation

FluentValidation was added to the DTO layer to enforce required fields and max lengths before the request reaches the service. Domain-level validations act as a second line of defense inside the entity factory method.

Concurrency and data integrity

EF Core's optimistic concurrency was considered but not implemented for this scope. The repository uses AsNoTracking() for reads and explicit UpdateAsync() for writes, which is sufficient for the current requirements.

Empty and boundary inputs

Tests cover: empty title, title exceeding 200 characters, due date set to yesterday, and Guid.Empty as userId. Each raises a DomainException with a descriptive message.